PainEngine Практический туториал

От первого проекта
до управляемого персонажа



2026.10.27.0 / 27.10.2026

Первый проект в лаунчере

1. Создать проект

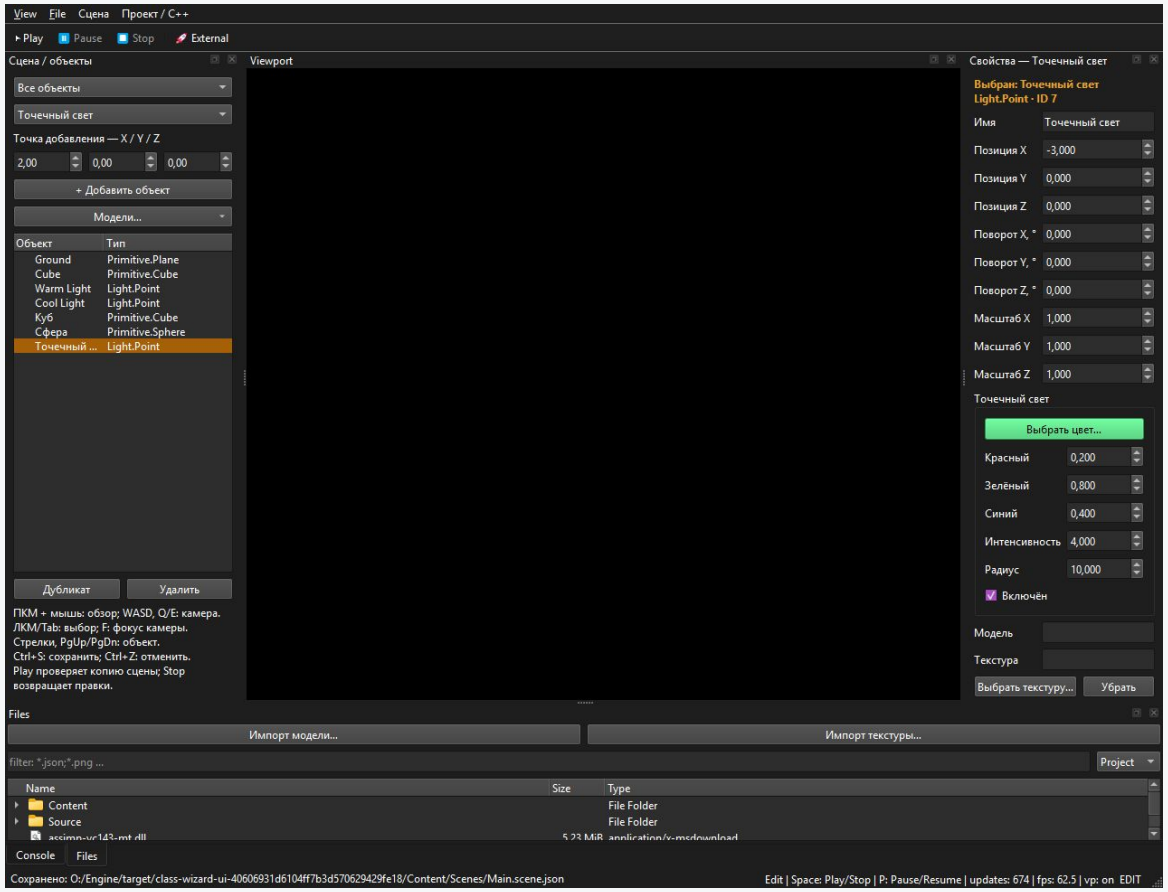
Выберите папку и название. Можно сгенерировать код C++ сразу, можно использовать шаблон

2. Редактировать

Выберите созданный проект в списке и откройте редактор.

Кириллица и пробелы допустимы в названии проекта. Для имени C++ класса действуют другие правила.

Окно редактора



Слева

Категория, тип объекта, точка добавления и иерархия сцены.

Справа

Свойства выбранного объекта: трансформация, цвет и параметры типа.

Сверху и снизу

Play / Pause / Stop, меню сцены, Files и Console.

Снимок панелей редактора. 3D-изображение viewport не попало в захват окна.

Камера и выделение в Edit

Действие	Управление
Движение камеры	WASD, вверх E, вниз Q
Обзор и ускорение	ПКМ + мышь, Shift ускоряет движение
Выбор объекта	ЛКМ в viewport или строка иерархии, Tab перебирает объекты
Фокус на выбранном	F
Перемещение объекта	Стрелки, Page Up / Page Down
Снять выделение	Esc

Сначала щёлкните viewport. При вводе текста команды движения не должны срабатывать.

Модели и текстуры

Импорт в проект

Меню «Сцена»: «Импорт модели…» или «Импорт текстуры…».

Размещение модели

«Добавить модель…» выбирает ресурс из Content/Models и размещает его в точке добавления.

Назначение текстуры

Выберите объект, затем текстуру в инспекторе или откройте её двойным щелчком в Files.

Модели: GLB, GLTF, OBJ, FBX. Текстуры: PNG, JPG, JPEG, BMP, TGA. Импорт моделей требует сборки с Assimp.

Свет, частицы и небо

Точечный свет

Добавьте источник, затем настройте цвет, интенсивность, радиус и флаг включения.

Искры и дым

Категория частиц содержит готовые варианты. Разместите эффект и настройте доступные свойства.

Базовое небо

В категории «Окружение» добавьте небо и проверьте сцену с нужной камеры.

Меняйте один параметр за раз, чтобы видеть его влияние на сцену.

Стандартный персонаж

Добавление «Игровые объекты», затем «Персонаж» и «Добавить объект».

Персонаж сам по себе не принимает ввод. Для управления нужно назначение в настройках Play.

Назначение для Play

Поле настроек Play	Что выбрать
Персонаж	Нужный объект из сцены
Камера	«Следовать за персонажем» для первого запуска
Стандартное управление	Включено: WASD и ПКМ + мышь
Расстояние и высота камеры	Начните со значений 5 и 2
Чувствительность обзора	Начальное значение 0,005

Откройте «Сцена / Настроить Play…». Диалог создаёт «Настройки Play», один объект на сцену.

Edit, Play, Pause и Stop

Режим / действие	Поведение
Edit	Редактирование рабочей сцены и свободная камера редактора
Play	Отдельная копия сцены, игровой контроллер и игровая камера
Pause	Приостановка симуляции и сброс активного игрового ввода
Stop	Возврат к рабочей сцене и камере редактора
External / «Играть»	Отдельный запуск игры по сохранённым данным проекта

В viewport: Space переключает Play / Stop, P переключает Pause / Resume.

Сохранение и структура проекта

Файл / папка	Назначение
Project.json	Пути к модулю, Content и стартовой сцене
Engine.lock.json	Закреплённая версия и конфигурация движка
Content/Scenes	Сцены с объектами, трансформациями и настройками Play
Content/Models, Content/Textures	Ресурсы проекта
Source и Build.ps1	Код и запуск сборки C++ проекта
Binaries	Собранные версии DLL проекта

Ctrl+S: «Сохранить и сделать стартовой». Для переноса сохраняйте всю папку проекта.

Граница между редактором и C++

Задача	Без кода	Через SDK / C++
Сцена и оформление	Объекты, модели, свет, эффекты	Свои компоненты и данные
Персонаж	Трансформация и скорость	Свой Character и поведение
Камера Play	Слежение или отдельная камера	Свой UpdateCamera / SceneCamera
Управление	Стандартные WASD и ПКМ	Действия, сочетания, обработчики
Каталог объектов	Встроенные типы	Регистрация своих классов

Контроллер создаётся игрой и не является отдельным добавляемым объектом сцены.

Создание C++ класса

1. Подключение кода — «Проект / C++»: «Добавить C++ в проект…». Нужны Visual Studio C++ tools и Windows SDK.

2. Мастер класса — «Добавить C++ класс…». Выберите Character, Component, Game или Object и задайте имя.

3. Сборка — «Собрать и открыть заново». После успешной проверки класс появится в доступном ему каталоге.

Допустимо: Character5454. Недопустимо: 5454, «Мой герой», Hero.h. Имя класса начинается с буквы или _.

Жизненный цикл пользовательского персонажа

```
void MyHero::BeginPlay()
{
    Character::BeginPlay();
    // Подписки и подготовка игровой логики
}

void MyHero::Tick(float deltaTime)
{
    Character::Tick(deltaTime);
    // Поведение персонажа в игре
}
```

BeginPlay вызывается один раз перед первой обработкой игрового ввода и Tick.

Сгенерированные методы сохраняют вызов непосредственного родителя.

Как ввод доходит до персонажа

Устройство и контекст

Клавиатура / мышь поступают в Input::System. Контексты: UI, Editor и Game.

Действие и обработчик

Привязка сопоставляет клавиши имени действия. Подписчик получает Pressed, Held, Released или Axis.

Контроллер и объект

CharacterController управляет назначением, движением Character и игровой камерой.

Обычный C++ класс тоже может подписаться. Наследование от GameComponent не требуется.

Подписка из обычного класса

```
class Commands {
    Input::Connection mark_;
    Input::Scope scope_;
public:
    Commands(Input::System& input) {
        input.Bind({"Mark", Input::Context::Game,
                    {Keys::Control, Keys::M}});
        mark_ = input.Subscribe(scope_,
            Input::Context::Game, "Mark",
            Input::Phase::Pressed,
            [this](const Input::Event&) { Mark(); });
    }
    void Mark();
};
```

Храните Connection как поле.

Scope принадлежит владельцу обработчика и отключает подписки при уничтожении.

Временное отключение: scope_.SetEnabled(false).

Сочетания и переназначение

```
// Тот же обработчик действия Mark
input.Unbind(Input::Context::Game, "Mark");
input.Bind({"Mark", Input::Context::Game,
        {Keys::Alt, Keys::K, Keys::J}});

// Две привязки одной оси
input.Bind({"MoveX", Input::Context::Game,
        {Keys::D}, 1});
input.Bind({"MoveX", Input::Context::Game,
        {Keys::A}, -1});
```

Все клавиши сочетания должны удерживаться одновременно.

По умолчанию более специфичная комбинация поглощает обычную клавишу.

A + D даёт ноль по оси.

Своя схема управления

```
class MyController : public CharacterController {
public:
    using CharacterController::CharacterController;
    void BindInput() override {
        game_.GetInput().Bind({"Go",
            Input::Context::Game, {Keys::I}});
        connections_.push_back(game_.GetInput().Subscribe(
            inputScope_, Input::Context::Game, "Go",
            Input::Phase::Axis,
            [this](const Input::Event& e) {
                forward_ = e.value;
            }));
    }
};
```

Этот вариант заменяет стандартные привязки: I задаёт движение вперёд.

Game выбирает контроллер через CreateCharacterController().

Для своей камеры переопределите UpdateCamera().

Обновление движка и проекта

Закреплённая версия	Проект использует выпуск из Engine.lock.json. SDK должен соответствовать версии и конфигурации.
Новый выпуск	В лаунчере добавьте версию и используйте «Обновить копию…». Проверьте полученную копию проекта.
Проект с C++	После обновления пересоберите DLL новым SDK и проверьте сцену и Play.

Изменение номера в Engine.lock.json вручную не заменяет обновление проекта.

Частые вопросы при первом запуске

Симптом	Что проверить
Персонаж не двигается	Play, фокус viewport, назначение и флажок управления
После Stop позиция прежняя	Это ожидаемо: Play изменяет временную копию
Не создаётся C++ класс	Имя начинается с буквы или _, без пробелов и .h
Новый класс не виден	Успешная сборка, регистрация и повторное открытие редактора
Ошибка сборки / загрузки	Console, CodeBuild.log и логи кандидата в Binaries
Пропала модель или текстура	Ресурс импортирован в Content и перенесён вместе с проектом